

Prisma ORM 6.7.0

Disciplina: Programação Web

Trabalho de Pesquisa

Alunos:

- Matheus Witte Ditz
 - Gabriel Rizzatto
-

1. Inicialização e Instalação de Dependências

Inicie o projeto Node.js: Dentro da pasta do seu projeto, crie o ficheiro `package.json` padrão.

```
npm init -y
```

Instale as dependências principais: Instale o Express (para criar a API), CORS (para permissões de acesso), PG (o driver do PostgreSQL) e o Prisma Client fixado na versão requerida.

```
npm install express cors pg @prisma/client@6.7.0
```

Instale as dependências de desenvolvimento: Instale o Nodemon (para reiniciar o servidor automaticamente) e a CLI do Prisma, também fixada na versão `6.7.0`.

```
npm install --save-dev nodemon prisma@6.7.0
```

Configure os scripts: Abra o ficheiro `package.json` gerado e adicione os atalhos de execução dentro do bloco `"scripts"`.

```
"scripts": {  
  "start": "node index.js",  
  "start:dev": "nodemon index.js"  
}
```

2. Configuração do Prisma e Base de Dados

Inicialize a estrutura do Prisma: Este comando cria a pasta `prisma` com o ficheiro `schema.prisma` e gera o ficheiro `.env` na raiz.

```
npx prisma init
```

Configure a ligação: Abra o ficheiro `.env` e defina as credenciais da sua base de dados PostgreSQL.

```
DATABASE_URL="postgresql://usuario:senha@localhost:5432/nome_do_banco?
schema=public"
```

Escreva os seus Models: No ficheiro `prisma/schema.prisma`, certifique-se de que o provider é `"postgresql"` e defina as suas tabelas (ex: `model Professor`, `model Aluno`, `model Turma`).

Exemplo de `schema.prisma`

```
// Configura o cliente Prisma gerado a partir deste schema.
generator client {
  // Define a biblioteca do cliente que o Prisma vai gerar.
  provider = "prisma-client-js"
  // Define o diretório de destino dos arquivos gerados.
  output   = "../generated/prisma"
}

// Configura a conexão com a base de dados.
datasource db {
  // Indica que a base de dados utilizada é PostgreSQL.
  provider = "postgresql"
  // Lê a URL de conexão da variável de ambiente DATABASE_URL.
  url      = env("DATABASE_URL")
}

// Define o model Professor, correspondente a uma tabela.
model Professor {
  // Chave primária inteira, gerada automaticamente em sequência.
  id      Int      @id @default(autoincrement())
  // Campo de texto para o nome do professor.
  nome    String
  // Campo de texto com valor único, sem duplicatas.
  email   String @unique
  // Relação de um professor com várias turmas.
  turmas  Turma[]
}

// Define o model Aluno, correspondente a uma tabela.
model Aluno {
  // Chave primária inteira, gerada automaticamente em sequência.
  id      Int      @id @default(autoincrement())
  // Campo de texto para o nome do aluno.
  nome    String
  // Matrícula única para cada aluno.
  matricula String @unique
}
```

```
// Relação de um aluno com várias turmas.
turmas    Turma[]
}

// Define o model Turma, correspondente a uma tabela.
model Turma {
  // Chave primária inteira, gerada automaticamente em sequência.
  id        Int        @id @default(autoincrement())
  // Campo de texto para o nome da turma.
  nome      String
  // Campo inteiro que armazena a chave estrangeira do professor.
  professorId Int
  // Relação com Professor: professorId referencia o campo id.
  professor Professor @relation(fields: [professorId], references: [id])
  // Relação de uma turma com vários alunos.
  alunos    Aluno[]
}
```

3. Explicação dos Arquivos Criados

- **.env**: Arquivo gerado na raiz do projeto destinado a armazenar credenciais e variáveis de ambiente confidenciais. É aqui que a **DATABASE_URL** (string de conexão com o PostgreSQL) fica guardada. Este arquivo nunca deve ser enviado para o GitHub.
- **prisma/schema.prisma**: O arquivo de configuração central do Prisma. Serve para definir qual banco de dados está a ser utilizado (bloco **datasource**) e para desenhar a estrutura de todas as tabelas (blocos **model**).
- **node_modules/.prisma/client**: Uma pasta oculta gerada dinamicamente sempre que executa o comando **npx prisma generate**. Contém o código autogerado que ensina o Node.js a interagir com as tabelas que você definiu no **schema.prisma**.
- **prisma/migrations/**: Pasta gerada automaticamente ao rodar a sua primeira migration. Guarda o histórico de todas as alterações feitas no banco de dados em arquivos **.sql**, servindo como uma linha do tempo da estrutura da base de dados.

4. Trabalhando com Migrations (Controle de Versão do Banco)

As migrations funcionam como um "histórico de commits" para o seu banco de dados. Em vez de alterar as tabelas diretamente de forma invisível, o Prisma gera arquivos SQL que documentam cada mudança (como criar uma tabela, adicionar uma coluna ou mudar um tipo de dado).

Criar e aplicar uma migration no ambiente de desenvolvimento: Após adicionar ou alterar um model no **schema.prisma**, execute este comando. Ele pede um nome descritivo, gera o arquivo **.sql** correspondente e aplica a mudança na base de dados.

```
npx prisma migrate dev --name cria_tabela_turmas
```

Aplicar migrations em produção: Quando o seu projeto for para um servidor real, não utilize o comando **dev**. Em vez disso, use o comando seguro que apenas aplica o histórico de arquivos **.sql** já existentes sem

risco de resetar dados:

```
npx prisma migrate deploy
```

Diferença entre `db push` e `migrate dev`: Use `npx prisma db push` apenas na prototipagem rápida, quando ainda está a testar a estrutura e não se importa em perder os dados ou não precisa de um histórico formal. Use `npx prisma migrate dev` quando a estrutura estiver mais estável e precisar de um histórico formal, contínuo e seguro das alterações.

5. Como Importar e Usar o Prisma

Para interagir com o banco de dados, é necessário instanciar o `PrismaClient`:

```
// Importa o cliente da biblioteca do Prisma
const { PrismaClient } = require("../generated/prisma/index.js");

// Cria a instância de conexão com a base de dados
const prisma = new PrismaClient();
```

6. Como Fazer um CRUD (Exemplo Prático)

Com o Prisma importado, pode executar as quatro operações fundamentais. Exemplo com uma entidade fictícia `Professor`:

```
// 1. CREATE (Criar um novo registo)
const criarProfessor = async (dados) => {
  return await prisma.professor.create({
    data: {
      nome: dados.nome,
      email: dados.email
    }
  });
};

// 2. READ (Buscar/Listar registos)
const listarProfessores = async () => {
  return await prisma.professor.findMany({
    include: { turmas: true } // O include traz os dados de tabelas relacionadas
  });
};

// 3. UPDATE (Atualizar um registo existente)
const atualizarProfessor = async (id, dados) => {
  return await prisma.professor.update({
    where: { id: Number(id) }, // O ID na cláusula where é sempre obrigatório e numérico
    data: {
```

```
        nome: dados.nome,
        email: dados.email
      }
    });
  };

// 4. DELETE (Apagar um registo)
const apagarProfessor = async (id) => {
  return await prisma.professor.delete({
    where: { id: Number(id) }
  });
};
```

7. Principais Comandos (Dia a Dia)

- **npx prisma format**: Organiza automaticamente a indentação e valida a sintaxe do seu `schema.prisma`, avisando se houver erros antes mesmo de tentar enviar para a base de dados.
- **npx prisma generate**: Lê o seu `schema.prisma` e atualiza a biblioteca `@prisma/client` instalada no passo 2. Obrigatório correr sempre que criar ou alterar tabelas, para que o Node.js reconheça as novidades.
- **npx prisma db push**: Sincroniza rapidamente as suas tabelas com o PostgreSQL, apagando ou criando o que for necessário sem gerar ficheiros de histórico (ótimo para a fase de desenvolvimento rápido).
- **npx prisma migrate dev**: Compara o código com a base de dados e gera um ficheiro `.sql` detalhado com o histórico de alterações (essencial para ambientes de produção e trabalho em equipa).
- **npx prisma studio**: Abre uma interface gráfica interativa no navegador (`localhost:5555`) para visualizar, inserir ou apagar registos diretamente nas tabelas.